# Multilayer Perceptron

Dinanath Padhya*, Krishna Acharya†
*THA078BEI010 †THA078BEI020
Department of Electronics, Communication and Information Engineering
Thapathali Campus, IOE, Tribhuvan University, Nepal

***Abstract*—**

***Index Terms*—**

## I. Introduction

### A. Background

### B. Literature Review

### C. Objectives

This laboratory experiment aims to:

- Objective 1

## II. Methodology

### A. Implementation Environment

## III. Theoretical Background

### A. Activation Functions

Activation functions are mathematical functions applied to the output of each neuron in a neural network layer. Their primary purpose is to introduce non-linearity into the model, enabling neural networks to approximate complex, non-linear mappings between inputs and outputs. Without activation functions, a neural network, regardless of its depth, would behave as a single linear transformation, severely limiting its representational power.

*1) Sigmoid:* The sigmoid function is a smooth, S-shaped curve that maps any real-valued input to the range $(0, c)$, making it especially suitable for binary classification tasks. The general form of the sigmoid function with parameters $a$, $b$, and $c$ is:

$$\sigma(x) = \frac{c}{a + be^{-x}} \tag{1}$$

where $a$, $b$, and $c$ are constants that control the curve's shape:

- **Parameter** $c$ sets the upper asymptote (maximum value) of the function.
- **Parameter** $a$ affects the lower asymptote; when $a = 1$, the function ranges from 0 to $c$.
- **Parameter** $b$ controls the steepness and direction; increasing $b$ makes the curve steeper, while negative $b$ reflects it horizontally.

The standard sigmoid, with $a = 1$, $b = 1$, $c = 1$, produces the classic S-curve from 0 to 1, crossing 0.5 at $x = 0$ (see Figure 1).

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \tag{2}$$

The sigmoid function is widely used in neural networks due to its differentiability and well-defined gradient, which is essential for backpropagation:

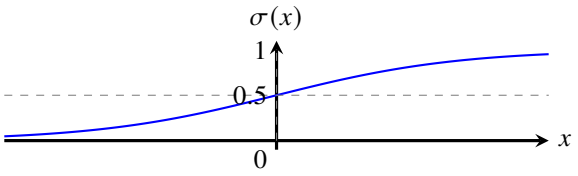



Fig. 1. Sigmoid Activation Function

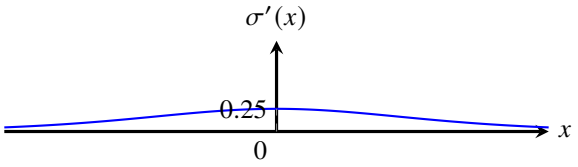



Fig. 2. Derivative of Sigmoid Activation Function: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$

*2) Tanh:* Tanh outputs values in $(-1, 1)$, centering the data and often leading to faster convergence than sigmoid:

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}} \tag{3}$$

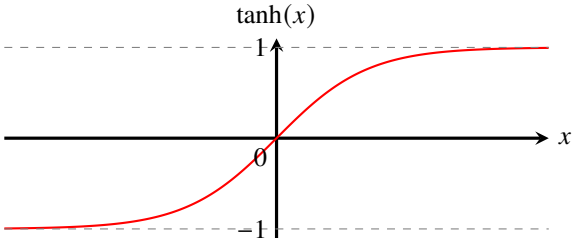



Fig. 3. Tanh Activation Function

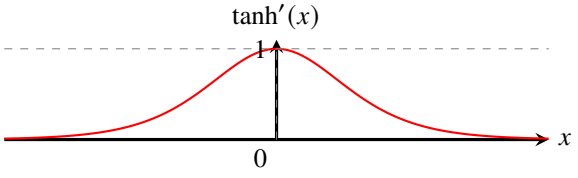



Fig. 4. Derivative of Tanh Activation Function: $\tanh'(x) = 1 - \tanh^2(x)$

Its derivative is:

$$\tanh'(x) = 1 - \tanh^2(x) \tag{4}$$

Like sigmoid, it can also suffer from vanishing gradients.

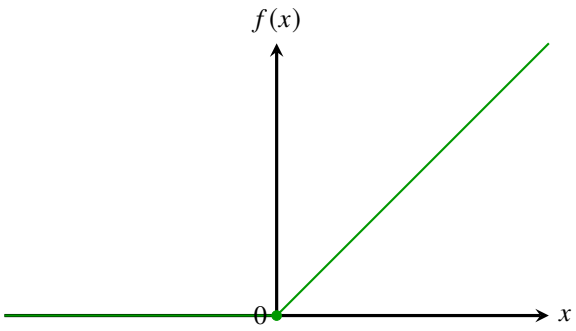



Fig. 5. ReLU Activation Function: $f(x) = \max(0, x)$

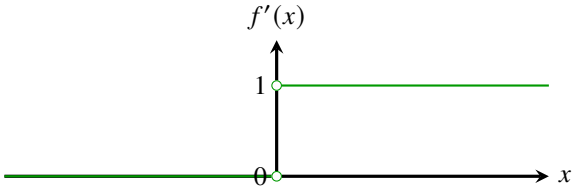



Fig. 6. Derivative of ReLU Activation Function

*3) ReLU (Rectified Linear Unit):* ReLU is widely used due to its simplicity and effectiveness:

$$f(x) = \max(0, x) \tag{5}$$

It outputs zero for negative inputs and the input itself for positive values. Its derivative is:

$$f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

ReLU helps mitigate the vanishing gradient problem but can lead to dead neurons if too many outputs become zero.

*4) Leaky ReLU:* This variant allows a small, non-zero gradient when the unit is not active:

$$f(x) = \max(\alpha x, x) \tag{7}$$

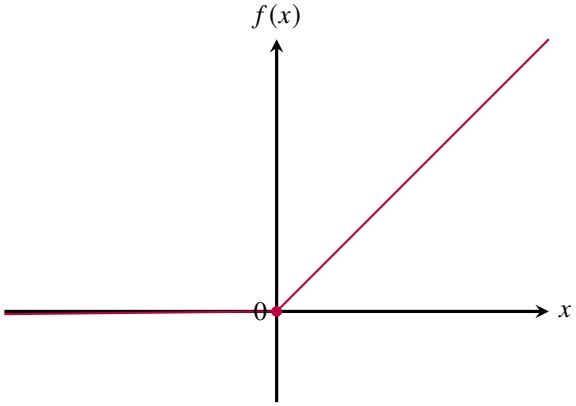



Fig. 7. Leaky ReLU Activation Function: $f(x) = \max(\alpha x, x)$ with $\alpha = 0.01$

Where $\alpha$ is a small constant (e.g., 0.01). This helps address the dead neuron issue.

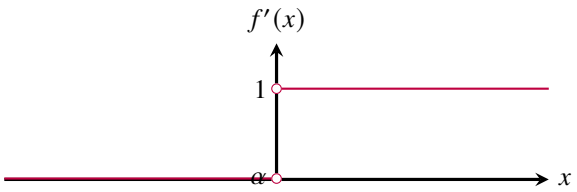



Fig. 8. Derivative of Leaky ReLU Activation Function: $f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ \alpha & \text{otherwise} \end{cases}$

*5) Softmax:* Used in the output layer for multi-class classification:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \tag{8}$$

Softmax converts logits into probabilities that sum to 1. Its derivative is more complex, involving the Jacobian matrix.

Activation functions are chosen based on the problem type and network architecture. Their derivatives are essential for gradient-based optimization, as they determine how errors are propagated backward through the network during training.

### *B. Pseudo-Random Number Generators (PRNGs)*

Pseudo-random number generators (PRNGs) are deterministic algorithms that produce sequences of numbers with properties approximating true randomness. While not truly random (as they're based on mathematical formulas), high-quality PRNGs produce sequences that pass statistical tests for randomness. In neural network development and machine learning broadly, PRNGs serve several critical functions:

- **Weight Initialization:** Neural networks require random initial weights to break symmetry between neurons. If all weights started with identical values, neurons in the same layer would compute identical outputs and gradients, making the network ineffective. Common initialization strategies like He, Xavier/Glorot, and standard normal distributions all rely on PRNGs.
- **Dataset Shuffling:** Random shuffling of training data prevents the network from learning artificial patterns based on the order of samples. This is particularly important in mini-batch gradient descent, where the composition of each batch significantly affects convergence.
- **Data Augmentation:** Random transformations (rotations, translations, noise addition) create variations of training examples, effectively expanding the dataset and improving model robustness.
- **Dropout Regularization:** This technique randomly deactivates neurons during training with probability $p$, requiring high-quality random number generation to be effective.
- **Stochastic Processes:** Many advanced ML techniques like stochastic gradient descent, Monte Carlo methods, and genetic algorithms rely on randomness for exploration of the parameter space.

- Initializing model weights randomly to break symmetry and ensure diverse learning.
- Shuffling datasets to prevent learning order bias.
- Generating synthetic data for experiments and augmentation.

`torch.rand` is a PyTorch function that returns a tensor filled with random numbers from a uniform distribution over $[0, 1)$. Since PRNGs are deterministic given a seed, setting the random seed `torch.manual_seed(42)` ensures reproducibility of experiments, which is vital for scientific research and debugging.''

### C. Encoding

Machine learning models require numerical input. For categorical data, encoding methods are used:

- **Label Encoding (LE):** Each unique category is assigned an integer value. For example, [cat, dog, fish] $\rightarrow$ [0, 1, 2]. This method is simple but can introduce unintended ordinal relationships between categories, which may mislead some models.
- **One-Hot Encoding (OHE):** Each category is represented as a binary vector with a single 1 and the rest 0s. For example, [cat, dog, fish] $\rightarrow$ [1,0,0], [0,1,0], [0,0,1]. OHE avoids ordinal relationships and is preferred for nominal categorical variables, especially in neural networks.

In classification tasks, especially with neural networks, one-hot encoding is often used for target labels. For example, in a three-class problem, the label 2 would be encoded as [0, 0, 1]. This format is compatible with loss functions like cross-entropy, which expect probability distributions as targets.

### D. Loss Functions

Loss functions, also known as cost or objective functions, measure the discrepancy between the model's predictions and the actual target values. They guide the optimization process by providing a signal (the loss) that the optimizer seeks to minimize.

*1) Mean Squared Error (MSE):*

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{9}$$

Used for regression, MSE penalizes larger errors more heavily. Its derivative with respect to $\hat{y}$ is:

$$\frac{\partial L}{\partial \hat{y}} = -2(y - \hat{y}) \tag{10}$$

*2) Cross-Entropy Loss:*

$$L = -\sum_{i} y_i \log(\hat{y}_i) \tag{11}$$

Used for classification, especially with one-hot encoded targets. It measures the dissimilarity between the true distribution $y$ and the predicted distribution $\hat{y}$. For binary classification, it reduces to the binary cross-entropy (log loss). Derivative of Cross-Entropy Loss Assuming the logits $\mathbf{z}$ produce $\hat{\boldsymbol{y}}$ via softmax, the derivative with respect to $z_i$ is:

$$\frac{\partial L}{\partial z_i} = \hat{y}_i - y_i.$$

*3) Negative Log Likelihood (NLL):* Often used with softmax for multi-class classification. It quantifies the likelihood of the observed data under the parameterized model.

The choice of loss function depends on the task (regression, binary classification, multi-class classification) and directly affects the learning dynamics. The derivatives of loss functions are used in backpropagation to compute gradients for parameter updates.

*4) Optimizers:* Optimizers are algorithms that adjust the parameters (weights and biases) of a neural network to minimize the loss function. They use the gradients computed during backpropagation to update parameters in the direction that reduces the loss.

Key optimizers include:

- **Stochastic Gradient Descent (SGD):** The basic optimizer. Parameters are updated as $w \leftarrow w - \eta \nabla L$, where $\eta$ is the learning rate. SGD can be noisy but helps escape local minima. Mini-batch SGD is commonly used for efficiency.
- **Momentum:** Accelerates SGD by adding a fraction of the previous update to the current one, helping to smooth out oscillations.
- **RMSprop:** Adapts the learning rate for each parameter by dividing the gradient by a running average of its recent magnitude. This helps in dealing with non-stationary objectives and speeds up convergence.
- **Adagrad:** Adapts learning rates based on the frequency of parameter updates, performing larger updates for infrequent parameters.
- **Adadelta:** An extension of Adagrad that seeks to reduce its aggressive, monotonically decreasing learning rate.
- **Adam:** Combines the benefits of RMSprop and momentum. Maintains running averages of both the gradients and their squares, and uses these to compute adaptive learning rates for each parameter. Adam is widely used due to its robustness and efficiency.

Optimizers are essential because manual updates are impractical for large models and datasets. The choice of optimizer affects convergence speed, stability, and final model performance. For example, Adam often converges faster and more reliably than vanilla SGD, especially in deep networks.

*5) Training and Evaluation Loops:* The process of training a neural network involves iteratively updating model parameters to minimize the loss. The standard pipeline consists of:

1) **Data Processing:** Data is cleaned, normalized, and encoded (e.g., one-hot encoding for labels).
2) **Model Forward Pass:** Input data is passed through the network to obtain predictions.
3) **Loss Calculation:** The loss function computes the error between predictions and true labels.
4) **Backpropagation:** The gradients of the loss with respect to each parameter are computed using the chain rule.
5) **Optimizer Step:** The optimizer updates the parameters using the computed gradients.
6) **Weight Update:** Parameters are adjusted to reduce the loss.

This sequence is repeated for multiple epochs over the training data. After training, the model is evaluated on a separate validation or test set to assess its generalization performance. Evaluation metrics (e.g., accuracy, precision, recall) are used to quantify performance.

*6) Backpropagation and Gradients:* Backpropagation is the core algorithm for training neural networks. It efficiently computes the gradients of the loss function with respect to each parameter by applying the chain rule of calculus through the network's layers.

For a simple logistic regression model:

1. Compute the derivative of the loss with respect to the output: $\frac{\partial L}{\partial \hat{y}}$
2. Compute the derivative of the output with respect to the input to the activation: $\frac{\partial \hat{y}}{\partial z}$
3. Compute the derivative of the input with respect to each weight: $\frac{\partial z}{\partial w_i}$

The total gradient for each weight is the product of these derivatives (chain rule):

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

This process is repeated for all parameters in the network. The gradients are then used by the optimizer to update the weights, gradually reducing the loss and improving the model's predictions. Backpropagation is efficient due to dynamic programming (caching intermediate results), making it feasible to train deep networks.

*Dataset*

## IV. Experiment & Model Implementation

## V. Results

## VI. Conclusion